

Documents Chunked Upload – Maintenance Reference

Oct 5, 2026 · @Mr G

Purpose and status

The module lets Documents accept files far larger than Odoo.sh will pass in one request, and keeps large PDFs from stalling on text indexing. It is version 19.0.1.0.3 and runs in production, verified with a 90 MB PDF on 2026-10-05.

Two problems drove it:

- **Platform request cap.** A single 90 MB upload request passed, but a 128 MB request was dropped by the Odoo.sh proxy before Odoo saw it (browser error `ERR_HTTP2_PROTOCOL_ERROR`, no matching line in `odoo.log`). The cap sits somewhere between those two sizes. It is observed, not documented, and raising Odoo's own limits cannot change it.
- **Slow PDF processing.** The standard `attachment_indexation` add-on reads every PDF page with `pdfminer` in pure Python to build a search index. For a 90 MB drawing, the final upload step spent 377 s in Python and about 1 s in the database.

The module works inside Odoo.sh and needs nothing on your office network or NAS.

How it works

Files above 60 MB are cut into 25 MB pieces in the browser, stored temporarily by the server, then joined and saved through Documents' own upload logic. Files at or below 60 MB take the standard Odoo route unchanged.

The large-file path, in order:

1. **Browser.** A patch replaces `uploadDocument` in the Documents service. Each piece goes to `/documents/upload_chunk`, with up to three tries and a growing pause. A notice shows "Uploading <name>: n%".
2. **Temporary storage.** Each piece is saved as a `documents.upload.chunk` record (upload id, user, sequence, data) in the database. Odoo's automatic cleanup removes abandoned pieces after 12 hours.

3. **Finish request.** When the last piece is in, the browser calls

`/documents/upload_finish` with an empty placeholder file plus the upload id, file name, type, piece count and total size. It goes through the normal file-upload service, so the Documents view refreshes as after any upload.

4. **Assembly.** The server checks that every piece is present and the total size matches, joins them into a temporary file, and passes it to Documents' `_documents_upload`. Folder placement, linking to a project, contact or fleet record, permissions and the chatter note are therefore the standard ones. The pieces are deleted afterwards.

The chunk routes accept internal users only, and the server (not the browser) rejects any file above the Documents upload limit.

PDF indexing. The module overrides `ir.attachment._index` to skip text extraction for PDFs above a size threshold (default 5 MB). This applies to every PDF attachment in Odoo, not only chunked uploads. Smaller PDFs are indexed as before. The module depends on `attachment_indexation` so its override runs first.

Diagnostics. The assembly and document-creation steps write `[chunked upload] ... took N s` lines to `odoo.log`. If either step runs longer than 30 s, the log also shows the current Python call stack every 30 s, which names the code that is slow.

Settings and parameters

Three system parameters control the behavior, and each must be a plain number with no commas, spaces or units. A value of `125,829,120` once made every page in production return an Internal Server Error.

Parameter	Unit	Production value	What it does
<code>document.max_fileupload_size</code>	bytes	524288000 (500 MB)	Documents upload limit. Both chunk routes reject larger files.

Parameter	Unit	Production value	What it does
<code>web.max_file_upload_size</code>	bytes	524288000 (500 MB)	Odoo's general upload limit, read by the web client on every page load.
<code>documents_chunked_upload.pdf_index_max_mb</code>	MB	not set (default 5)	PDFs above this size are saved without a text index. <code>0</code> skips all PDFs.

Three values live in the code, not in parameters, so changing them needs a code push: the 60 MB chunking threshold, the 25 MB piece size and the three retries, all at the top of `static/src/document_service_patch.js`.

To change a parameter, run this in the Shell of the right branch (check `print(env.cr.dbname)` first):

```
env['ir.config_parameter'].sudo().set_param('document.max_fileupload_size',
'524288000')
env['ir.config_parameter'].sudo().set_param('web.max_file_upload_size',
'524288000')
env.cr.commit()
env.registry.clear_all_caches()
print(env['documents.document'].get_document_max_upload_limit())
```

The last line should print the new value. A Shell change may not reach the web workers that serve users, because they keep the old value cached. After a Shell change, save the same parameters once through Settings > Technical > System Parameters (digits only), or restart production by pushing a trivial commit. Editing the database with raw SQL does not refresh the workers either.

To get the byte value of a size, multiply megabytes by 1,048,576.

Install, update and deploy

The module is the folder `documents_chunked_upload` at the top level of the Odoo.sh GitHub repository. It depends on `documents` and `attachment_indexation`, both already installed. Always test on staging first, then repeat on production.

To deploy a new version:

1. Raise the version number in `__manifest__.py` (currently 19.0.1.0.3).
2. In production only: choose a quiet time, tell users, and take a manual backup from the production branch's Backups tab.
3. Replace the module folder in your local clone with the new one (one copy, not nested), commit it on the target branch in GitHub Desktop, and push. Wait for the Odoo.sh build to turn green.
4. In Odoo, turn on developer mode, open Apps, click Update Apps List, remove the "Apps" filter and search for Documents Chunked Upload. Click Install, or Upgrade if it is already installed.
5. Check the two upload limits and the PDF threshold in the Shell (see the settings section), and reload the page with Ctrl+Shift+R.
6. Upload one large PDF yourself and watch `odoo.log` for errors before telling users.

Pushing to production rebuilds it and briefly interrupts users. Never push or drag staging onto production by accident: bring production's state into staging by merging `main` into `staging`, not the other way around.

Staging versus production

Staging and production behave differently in ways that already cost time during setup. These are observed behaviors, not Odoo.sh documentation.

Situation	What happens	What to do
Push to the staging branch	Each build appeared to start from a fresh copy of production data. Shell changes and the module install made in staging were gone after every push.	Repeat the setup after each push: check the limits, install or upgrade the module. Batch fixes into one push.

Situation	What happens	What to do
Which database am I in?	The Shell attaches to the branch you opened it from.	Run <code>print(env.cr.dbname)</code> first. Production's name contains <code>main</code> , staging's contains <code>staging</code> .
Parameter changed in the Shell	The Shell shows the new value, but running web workers kept the old one. Production uploads failed at 0% until it propagated.	Save the parameter through System Parameters (digits only), or restart by pushing a trivial commit.
Bad parameter value	A comma in <code>web.max_file_upload_size</code> broke every production page with an Internal Server Error.	Fix with <code>set_param</code> in the Shell, then restart if the error persists. Never type units or commas.
Cloud connector in staging	Staging is neutralized (mail and scheduled jobs off), but the Google Drive connector's behavior in staging is unverified.	Test with throwaway files only.
Two heavy uploads at once	A large PDF and an MP4 uploaded together once took staging down. Not retested since the PDF indexing fix.	Test concurrent uploads on a quiet production before telling all users.

To bring staging in line with production, merge `main` into `staging` in GitHub Desktop (current branch `staging`) and push, or restore a production backup into staging from the Backups tab. Check the destination branch before every restore.

Troubleshooting

Start with the browser's Network tab (F12) and the Odoo.sh Logs tab (`odoo.log`). Each symptom below was seen during setup.

Symptom	Likely cause	What to check or do
"Uploading ...: 0%" never moves, then "Upload of ... failed. Please try again."	The server refused the first piece. Most often the upload limit is lower than the file, or web workers still hold the old cached limit.	Filter Network for <code>upload_chunk</code> and read the status. 400: size or limit, so check both parameters and refresh workers. 403: not an internal user. 404: module not installed or loaded in this build. 500: read the traceback in <code>odoo.log</code> .

Symptom	Likely cause	What to check or do
"Some files could not be uploaded (max size: 67M)"	Odoo's standard message, so the module is not installed in this build, or <code>web.max_file_upload_size</code> is still 64 MB.	Check Apps for the module and version. Check both limits.
Every page shows Internal Server Error after editing a parameter	A parameter holds a non-numeric value such as <code>125,829,120</code> .	Look for <code>invalid web.max_file_upload_size</code> in <code>odoo.log</code> . Fix with <code>set_param</code> in the Shell, then restart.
Upload reaches 100% but "Processing.." lasts minutes	The final step is slow, usually PDF text indexing.	Search <code>odoo.log</code> for <code>[chunked upload]</code> . Stack lines naming <code>_index_pdf</code> or <code>pdfminer</code> mean indexing: lower <code>documents_chunked_upload.pdf_index_max_mb</code> . Any other add-on in the stack is the new suspect.
<code>ERR_HTTP2_PROTOCOL_ERROR</code> and no log line	A single request over the platform cap, so the file did not take the chunked path.	Confirm the module is installed and the browser was reloaded (Ctrl+Shift+R).
Large file in a folder upload fails	The third-party folder-upload app may post straight to <code>/documents/upload</code> and bypass the chunking.	Untested. Try one large and one small file in a test folder before relying on it.
Log message: PostgreSQL process terminated after 990 seconds	Odoo.sh killed a database session that sat idle too long, often a stuck Shell or an abandoned upload request.	Harmless alone. If uploads stall, restart the branch by pushing a trivial commit.

Known limits and open items

The largest files tested so far are a 190 MB MP4 and a 90 MB PDF, so anything bigger is unproven.

- **Memory.** The server holds the whole assembled file in memory while saving it, roughly two to three times its size. PDFs of 200 to 300 MB and the real ceiling have not been tested.
- **Concurrency.** A PDF and an MP4 uploaded together once took staging down, before the indexing fix. Not retested.
- **Search.** PDFs above the threshold (default 5 MB) cannot be found by words inside them. They are still found by name, folder and linked record.
- **Folder-upload app.** The third-party Upload Directory/Folder app has not been tested with large files.
- **No resume.** Closing the tab mid-upload loses it. Leftover pieces are removed after 12 hours, and the user starts the upload again.
- **Progress display.** The percentage moves once per 25 MB piece. The final "Processing..." step shows no percentage.

- **Users.** Chunked upload works for internal users only.
- **Platform cap.** The exact cap is unknown (between 90 and 128 MB). The 60 MB threshold leaves a safe margin.
- **Cloud connector.** How it handles files after a chunked upload has not been examined.
- **Odoo upgrades.** The module patches Enterprise code (`DocumentService.uploadDocument` , `ShareRoute._documents_upload`) and `attachment_indexation` . Re-test after any Odoo version change.

Before deploying any change, test on staging: a small file, a large PDF, a large video, two uploads at once, a folder upload, and an upload from a Project and a Contact to confirm it appears on that record's Documents button.

Files and version history

The module has five working files. On Odoo.sh the repository folder appears under `src/user/documents_chunked_upload` .

File	Purpose
<code>__manifest__.py</code>	Name, version, dependencies (<code>documents</code> , <code>attachment_indexation</code>), browser asset list.
<code>controllers/chunked_upload.py</code>	The <code>/documents/upload_chunk</code> and <code>/documents/upload_finish</code> routes, plus the <code>[chunked upload]</code> diagnostics.
<code>models/upload_chunk.py</code>	Temporary <code>documents.upload.chunk</code> records that hold the pieces.
<code>models/ir_attachment.py</code>	Skips PDF text indexing above the size threshold.
<code>static/src/document_service_patch.js</code>	Browser side: 60 MB threshold, 25 MB pieces, retries, progress notices, final request.

Version history, newest first (all dated 2026-10-05):

Version	Change
19.0.1.0.3	Skips PDF text indexing above 5 MB (adjustable). Adds the <code>attachment_indexation</code> dependency.
19.0.1.0.2	Adds timing and call-stack logging to the final upload step.
19.0.1.0.1	Removes the browser-side size pre-check. The server enforces the limit.
19.0.1.0.0	First version: chunked upload with a browser-side pre-check.